Docker Scenario Based questions part-2 (Dockerfile)

1. Image Layer Optimization in a Startup

A startup notices that their Docker images are extremely large and builds are slow.
Design a Dockerfile strategy using proper layer ordering to reduce image size and build time.
Students must demonstrate:
- Creating a Dockerfile
- Using FROM, RUN, COPY correctly
- Inspecting layers using docker history

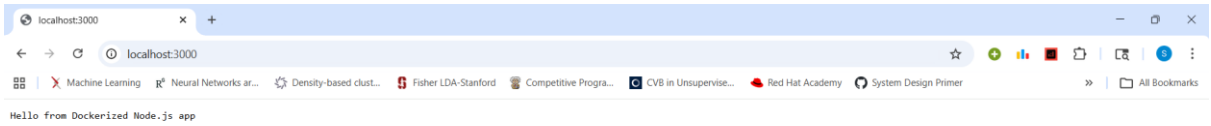
Implementation commands example:
docker build -t optimizedapp:v1 .
docker history optimizedapp:v1

```
Command Prompt - docker r
C:\Users\sreev>cd dockerfilepractice

C:\Users\sreev\dockerfilepractice>mkdir q1

C:\Users\sreev\dockerfilepractice>cd q1

C:\Users\sreev\dockerfilepractice\q1>notepad node.js

C:\Users\sreev\dockerfilepractice\q1>notepad package.json

C:\Users\sreev\dockerfilepractice\q1>notepad .dockerignore

C:\Users\sreev\dockerfilepractice\q1>notepad dockerfile

C:\Users\sreev\dockerfilepractice\q1>ren dockerfile.txt dockerfile
```

```
Command Prompt - docker r
C:\Users\sreev\dockerfilepractice\q1>dir
 Volume in drive C is OS
 Volume Serial Number is 0CE8-3131

 Directory of C:\Users\sreev\dockerfilepractice\q1

19-04-2026  14:49    <DIR>          .
19-04-2026  14:25    <DIR>          ..
19-04-2026  14:45                 57 .dockerignore
19-04-2026  14:49                125 dockerfile
19-04-2026  14:37                273 node.js
19-04-2026  14:44                194 package.json
               4 File(s)            649 bytes
               2 Dir(s)  21,938,860,032 bytes free

C:\Users\sreev\dockerfilepractice\q1>ren node.js server.sj

C:\Users\sreev\dockerfilepractice\q1>ren server.sj server.js
```

```
Command Prompt - docker r
C:\Users\sreev\dockerfilepractice\q1>docker build -t optimizedimg:v1 .
[+] Building 2.2s (10/10) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from dockerfile                                                  0.1s
 => => transferring dockerfile: 163B                                                                  0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                     1.3s
 => [internal] load .dockerignore                                                                     0.0s
 => => transferring context: 97B                                                                      0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c  0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c  0.1s
 => [internal] load build context                                                                     0.1s
 => => transferring context: 410B                                                                     0.1s
 => CACHED [2/5] WORKDIR /app                                                                         0.0s
 => CACHED [3/5] COPY package*.json .                                                                 0.0s
 => CACHED [4/5] RUN npm install --only=production                                                    0.0s
 => [5/5] COPY . .                                                                                    0.1s
 => exporting to image                                                                                0.5s
 => => exporting layers                                                                               0.1s
 => => exporting manifest sha256:b2b98fb8def6dc81ff0eb5ee5ab0e4719d64769b513fc2c8035c32827fd1357a     0.0s
 => => exporting config sha256:159a3c98ccaa327ed695e89221f646bc7d850b91395ed9d79139a53ab4734413       0.0s
 => => exporting attestation manifest sha256:48b4900941e5af48040497275b5f3ad7b5be7b8d1b6016604feeb3b993881834  0.0s
 => => exporting manifest list sha256:a8e49f18845574c9edeb97f49754c47b98e8074d6a65ccc85231b3c0320cbd5d  0.0s
 => => naming to docker.io/library/optimizedimg:v1                                                    0.0s
 => => unpacking to docker.io/library/optimizedimg:v1                                                 0.1s
```

```
C:\Users\sreev\dockerfilepractice\q1>docker run -p 3000:3000 --name optimizedapp1 optimizedimg:v1
server running on port 3000
```




```
Command Prompt

C:\Users\sreev\dockerfilepractice\q1>type dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json .
RUN npm install --only=production
COPY . .
CMD ["node","server.js"]
C:\Users\sreev\dockerfilepractice\q1>type server.js
const http=require('http')
const PORT=3000
const server = http.createServer((req,res)=> {
res.writeHead(200,{'Content-type':'text/plain'});
res.end('Hello from Dockerized Node.js app\n');
});
server.listen(PORT,()=>{
console.log(`server running on port ${PORT}`);
});
C:\Users\sreev\dockerfilepractice\q1>type package.json
{
"name":"docker-optimized-webapp",
"version":"1.0.0",
"description":"Simple node.js file for practical",
"main":"server.js",
"scripts":{
"start":"node server.js"
},
"dependencies":{}
}
C:\Users\sreev\dockerfilepractice\q1>
```

2. Secure Build Context Management

A development team accidentally includes sensitive files (.env, credentials) in Docker images.

Design a solution using .dockerignore to prevent this.

Students must:

- Create .dockerignore file
- Build image
- Verify excluded files are not copied

Commands example:

docker build -t secureimage:v1 .

```
C:\Users\sreev\dockerfilepractice\q1>dir
 Volume in drive C is OS
 Volume Serial Number is 0CE8-3131

 Directory of C:\Users\sreev\dockerfilepractice\q1

19-04-2026  15:51    <DIR>          .
19-04-2026  14:25    <DIR>          ..
19-04-2026  14:45                 57 .dockerignore
19-04-2026  15:51                 35 .env
19-04-2026  14:53                126 dockerfile
19-04-2026  14:44                194 package.json
19-04-2026  14:56                275 server.js
               5 File(s)            687 bytes
               2 Dir(s)  20,290,781,184 bytes free

C:\Users\sreev\dockerfilepractice\q1>
```

```
Command Prompt

C:\Users\sreev\dockerfilepractice\q1>notepad .env

C:\Users\sreev\dockerfilepractice\q1>docker build -t insecureimg .
[+] Building 3.2s (11/11) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from dockerfile                                                                0.1s
 => => transferring dockerfile: 163B                                                                                0.1s
 => [internal] load metadata for docker.io/library/node:18-alpine                                                   2.3s
 => [auth] library/node:pull token for registry-1.docker.io                                                         0.0s
 => [internal] load .dockerignore                                                                                   0.1s
 => => transferring context: 97B                                                                                    0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.1s
 => [internal] load build context                                                                                   0.1s
 => => transferring context: 192B                                                                                   0.1s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY package*.json .                                                                               0.0s
 => CACHED [4/5] RUN npm install --only=production                                                                  0.0s
 => [5/5] COPY . .                                                                                                  0.0s
 => exporting to image                                                                                              0.3s
 => => exporting layers                                                                                             0.1s
 => => exporting manifest sha256:84669a3d586f546a40a5ed6487d46151d27a04aadbeebf9490453d217750478c                   0.0s
 => => exporting config sha256:d2ee3d6e876a0c6185f01ed2dc5a8dad08ecd2a8dd1e3082c7c8180fc429822f                     0.0s
 => => exporting attestation manifest sha256:282ce70128952c233e477908757c5899b8a79caabaaa8f17c472b6ab95b39583        0.0s
 => => exporting manifest list sha256:ccdc3abaa3b40c87dca70d297b74ed54c85d99f9373c7231cf4f38cf9899d767               0.0s
 => => naming to docker.io/library/insecureimg:latest                                                               0.0s
 => => unpacking to docker.io/library/insecureimg:latest                                                            0.1s

C:\Users\sreev\dockerfilepractice\q1>docker run -it -p 3000:3000 --name insecapp insecureimg
server running on port 3000

C:\Users\sreev\dockerfilepractice\q1>ls
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\sreev\dockerfilepractice\q1>dir
 Volume in drive C is OS
 Volume Serial Number is 0CE8-3131

 Directory of C:\Users\sreev\dockerfilepractice\q1
```

-Above pic shows the .env file containing sensitive information has been included in the Docker image due to the COPY . . instruction, which is a security risk

```
Command Prompt - docker e
C:\Users\sreev\dockerfilepractice\q1>type .dockerignore
.env
node_modules
.git
.gitignore
Dockerfile
npm-debug.log
C:\Users\sreev\dockerfilepractice\q1>docker build -t secureimg .
[+] Building 2.9s (11/11) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from dockerfile                                                                                0.1s
 => => transferring dockerfile: 163B                                                                                                0.1s
 => [internal] load metadata for docker.io/library/node:18-alpine                                                                   2.1s
 => [auth] library/node:pull token for registry-1.docker.io                                                                         0.0s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 103B                                                                                                   0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e            0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e            0.0s
 => [internal] load build context                                                                                                   0.0s
 => => transferring context: 193B                                                                                                   0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                       0.0s
 => CACHED [3/5] COPY package*.json .                                                                                               0.0s
 => CACHED [4/5] RUN npm install --only=production                                                                                  0.0s
 => [5/5] COPY . .                                                                                                                  0.0s
 => exporting to image                                                                                                              0.3s
 => => exporting layers                                                                                                             0.1s
 => => exporting manifest sha256:eb00cd18c7d91fb1b19b2ba3dceeac681c0f9d2a3c33c02ead83697f0087bae8                                   0.0s
 => => exporting config sha256:dbc9378f6c08058e1311d700bbe4bf1f6e31086715d2a884bfa973aaec651be6                                    0.0s
 => => exporting attestation manifest sha256:3fd5f987d8ed4aab2123329b7292856c346911a95152504016bc6fcb7cb82119                      0.1s
 => => exporting manifest list sha256:c1ffc9e3f33c0253949b4876371e5a1dccfc4eafa14ec4ace0f1d452c7312283                             0.0s
 => => naming to docker.io/library/secureimg:latest                                                                                 0.0s
 => => unpacking to docker.io/library/secureimg:latest                                                                              0.0s

C:\Users\sreev\dockerfilepractice\q1>docker run -dit -p 3000:3000 --name secureapp secureimg
e30b420d3f8f359785e9fd356858269819cba6b63190595e45934908f5ca8c6b

C:\Users\sreev\dockerfilepractice\q1>docker exec -it secureimg sh
Error response from daemon: No such container: secureimg

C:\Users\sreev\dockerfilepractice\q1>docker exec -it secureapp sh
/app # ls -a
```

-The above pic shows the .env file is not present in the container as it is excluded from the build context via .dockerignore, ensuring no sensitive data is baked into the image.

```
C:\Users\sreev\dockerfilepractice\q1>docker history secureimg
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
c1ffc9e3f33c   10 minutes ago   CMD ["node" "server.js"]                        0B        buildkit.dockerfile.v0
<missing>      10 minutes ago   COPY . . # buildkit                             20.5kB    buildkit.dockerfile.v0
<missing>      16 minutes ago   RUN /bin/sh -c npm install --only=production…   28.7kB    buildkit.dockerfile.v0
<missing>      16 minutes ago   COPY package*.json . # buildkit                 12.3kB    buildkit.dockerfile.v0
<missing>      12 days ago      WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      12 months ago    CMD ["node"]                                    0B        buildkit.dockerfile.v0
<missing>      12 months ago    ENTRYPOINT ["docker-entrypoint.sh"]             0B        buildkit.dockerfile.v0
<missing>      12 months ago    COPY docker-entrypoint.sh /usr/local/bin/ # …   20.5kB    buildkit.dockerfile.v0
<missing>      12 months ago    RUN /bin/sh -c apk add --no-cache --virtual …   5.47MB    buildkit.dockerfile.v0
<missing>      12 months ago    ENV YARN_VERSION=1.22.22                        0B        buildkit.dockerfile.v0
<missing>      12 months ago    RUN /bin/sh -c addgroup -g 1000 node     && …    122MB     buildkit.dockerfile.v0
<missing>      12 months ago    ENV NODE_VERSION=18.20.8                        0B        buildkit.dockerfile.v0
<missing>      14 months ago    CMD ["/bin/sh"]                                 0B        buildkit.dockerfile.v0
<missing>      14 months ago    ADD alpine-minirootfs-3.21.3-x86_64.tar.gz /…   8.5MB     buildkit.dockerfile.v0
```

3. Container Startup Behavior

A container must always run a specific application when started.
Design a Dockerfile using CMD and ENTRYPOINT so the application starts automatically.
Students should:
- Write Dockerfile
- Run container
Commands example:
docker run myapp:v1

```
C:\Users\sreev\dockerfilepractice\q3>docker build --no-cache -t entrypracta .
[+] Building 4.4s (9/9) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from dockerfile                                                                                  0.7s
 => => transferring dockerfile: 138B                                                                                                  0.7s
 => [internal] load metadata for docker.io/library/python:3.11-alpine                                                                 3.3s
 => [auth] library/python:pull token for registry-1.docker.io                                                                         0.0s
 => [internal] load .dockerignore                                                                                                     0.1s
 => => transferring context: 2B                                                                                                       0.1s
 => [1/3] FROM docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61be52487693f54be7f1021647751ff365795703          0.1s
 => => resolve docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61be52487693f54be7f1021647751ff365795703          0.1s
 => [internal] load build context                                                                                                     0.1s
 => => transferring context: 28B                                                                                                      0.1s
 => CACHED [2/3] WORKDIR /app                                                                                                         0.0s
 => [3/3] COPY app.py .                                                                                                               0.1s
 => exporting to image                                                                                                                0.4s
 => => exporting layers                                                                                                               0.1s
 => => exporting manifest sha256:0b1ef7cd37cac1834603338af8cb3c4f739cdd80ca3a3a0b129b505d0430583c                                     0.0s
 => => exporting config sha256:283ef86fe2b7f5f66d0b6ee348b0c454437228d9aba5579d572e4dbfb7406c69                                       0.0s
 => => exporting attestation manifest sha256:f063e94b77aff2765064c62d0223d908a118cdf87c633407e55948abc56ba79b                         0.1s
 => => exporting manifest list sha256:f7654af9b894c2101705753b5d2a4e1579a8680cf4d086cb589a4c795b2c7a26                                0.0s
 => => naming to docker.io/library/entrypracta:latest                                                                                 0.0s
 => => unpacking to docker.io/library/entrypracta:latest                                                                              0.1s
```

```
C:\Users\sreev\dockerfilepractice\q3>docker run -it --name entryapp1 entrypracta
Container started successfully
Argument received:['app.py', 'default']
```

```
C:\Users\sreev\dockerfilepractice\q3>docker run -it --name entryapp entrypracta hi
Container started successfully
Argument received:['app.py', 'hi']

C:\Users\sreev\dockerfilepractice\q3>type app.py
import sys
print("Container started successfully")
if len(sys.argv)>1:
        print(f"Argument received:{sys.argv}")
else:
        print("No Argument provided")

C:\Users\sreev\dockerfilepractice\q3>
```

4. Environment Configuration for Applications

A company needs to configure application settings like database URL and environment mode inside containers.

Design a Dockerfile solution using ENV variables.

Students must:

- Define environment variables

- Run container and verify them

Commands example:

docker run -e APP_ENV=production myapp:v1

```
Command Prompt

 Directory of C:\Users\sreev\dockerfilepractice\q4

19-04-2026  17:32    <DIR>          .
19-04-2026  17:23    <DIR>          ..
19-04-2026  17:26               185 app.py
19-04-2026  17:32               135 dockerfile
               2 File(s)            320 bytes
               2 Dir(s)  20,669,685,760 bytes free

C:\Users\sreev\dockerfilepractice\q4>type app.py
import os

print("App started")

app_env=os.getenv("APP_ENV","development")
db_url=os.getenv("DB_URL","not_set")

print(f"Environment:{app_env}")
print(f"Database URL:{db_url}")
C:\Users\sreev\dockerfilepractice\q4>docker build -t myenvapp:1.0 .
[+] Building 3.6s (9/9) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from dockerfile                                                                0.4s
 => => transferring dockerfile: 174B                                                                                0.3s
 => [internal] load metadata for docker.io/library/python:3.11-alpine                                               2.5s
 => [auth] library/python:pull token for registry-1.docker.io                                                       0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/3] FROM docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61be52487693f54be7f1021647751ff365795703      0.1s
 => => resolve docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61be52487693f54be7f1021647751ff365795703      0.0s
 => [internal] load build context                                                                                   0.1s
 => => transferring context: 220B                                                                                   0.0s
 => CACHED [2/3] WORKDIR /app                                                                                       0.0s
 => [3/3] COPY app.py .                                                                                             0.1s
 => exporting to image                                                                                              0.4s
 => => exporting layers                                                                                             0.1s
 => => exporting manifest sha256:9a0b2665962738e853f4bb2983288a298a7e469a3ab093a7df9e66f590b79212                   0.0s
 => => exporting config sha256:818067b6058a63b164e46f46f7b084e18919aa7aa6465a982450dab36456b149                     0.0s
 => => exporting attestation manifest sha256:336d5b99e38e83481d60da82c3b7cc5fb8a1940de00949ed69f55b5835ba1088        0.1s
 => => exporting manifest list sha256:3f7ad8cdd6cc859c049a3aeabd6b9385b1557176c7284586cadf035d0a01de21              0.0s
```

```
C:\Users\sreev\dockerfilepractice\q4>docker run myenvapp:1.0
App started
Environment:production
Database URL:localhost:5342

C:\Users\sreev\dockerfilepractice\q4>docker run -e APP_ENV=development -e DB_URL=dev-db:5432 myenvapp:1.0
App started
Environment:development
Database URL:dev-db:5432
```

```
C:\Users\sreev\dockerfilepractice\q4>type dockerfile
FROM python:3.11-alpine
WORKDIR /app
ENV APP_ENV=production
ENV DB_URL=localhost:5342
COPY app.py .
ENTRYPOINT ["python","app.py"]
C:\Users\sreev\dockerfilepractice\q4>
```

5. Application Port Exposure

A web application container must be accessible from the host machine.
Design a solution using EXPOSE and port mapping.
Students must:
- Define EXPOSE in Dockerfile
- Run container with port mapping
Commands example:
docker run -p 8080:80 webapp:v1

```
Terminal
PS C:\Users\sreev\dockerfilepractice\q5> notepad server.js
PS C:\Users\sreev\dockerfilepractice\q5> docker build -t webapp:p1 .
[+] Building 2.5s (8/8) FINISHED                                   docker:desktop-linux
 => [internal] load build definition from dockerfile                                0.1s
 => => transferring dockerfile: 127B                                                0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                   1.5s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.1s
 => [internal] load build context                                                   0.1s
 => => transferring context: 309B                                                   0.1s
 => CACHED [2/3] WORKDIR /app                                                       0.0s
RAM 1.08 GB  CPU 0.08%   Disk: 48.88 GB used (limit 1006.85 GB)
Terminal   Update available
```

```
Terminal
 => CACHED [2/3] WORKDIR /app                                                       0.0s
 => [3/3] COPY server.js .                                                          0.1s
 => exporting to image                                                              0.5s
 => => exporting layers                                                             0.2s
 => => exporting manifest sha256:102f2e565a6886719fcd7c2c1b6294958a08d6081f8d05eeafb1146f5e9d555d   0.0s
 => => exporting config sha256:77ab5ac51a4f80cff9de81c8d92f5d22ac4e988204d2e20593be842462be4150   0.0s
 => => exporting attestation manifest sha256:5b43bd04076202bb2dfb81699e8fd079bf6258e6f2c3abc3d27ac3cfe629c3fc   0.1s
 => => exporting manifest list sha256:80eb12dbc26f91f7cadc4587b0881b93417226287eb47b0423f869870cd676ca   0.0s
 => => naming to docker.io/library/webapp:p1                                        0.0s
 => => unpacking to docker.io/library/webapp:p1                                     0.1s
PS C:\Users\sreev\dockerfilepractice\q5> docker run -p 3000:3000 --name webpract1 webapp:p1
Server running on 3000
RAM 1.09 GB  CPU 0.25%   Disk: 48.88 GB used (limit 1006.85 GB)
Terminal   Update available
```

localhost:3000

Hello from port exposure practical

```
Terminal
PS C:\Users\sreev\dockerfilepractice\q5> type dockerfile
FROM node:18-alpine
WORKDIR /app
COPY server.js .
EXPOSE 3000
CMD ["node","server.js"]
PS C:\Users\sreev\dockerfilepractice\q5> type server.js
const http=require('http')
const PORT=3000
const server=http.createServer((req,res)=>{
        res.writeHead(200,{'Content-type':'text/plain'});
        res.end('Hello from port exposure practical');
        });
RAM 1.10 GB  CPU 0.25%   Disk: 48.88 GB used (limit 1006.85 GB)
Terminal   Update available
```

```
Terminal
EXPOSE 3000
CMD ["node","server.js"]
PS C:\Users\sreev\dockerfilepractice\q5> type server.js
const http=require('http')
const PORT=3000
const server=http.createServer((req,res)=>{
        res.writeHead(200,{'Content-type':'text/plain'});
        res.end('Hello from port exposure practical');
        });
server.listen(PORT,()=>{
        console.log(`Server running on ${PORT}`);
        });
PS C:\Users\sreev\dockerfilepractice\q5>
RAM 1.10 GB  CPU 0.17%   Disk: 48.88 GB used (limit 1006.85 GB)
Terminal   Update available
```

6. Container Networking for Microservices

A microservice architecture requires frontend and backend containers to communicate internally.

Design a networking solution using bridge networks and DNS.

Students must:

- Create network
- Run two containers
- Verify connectivity

Commands example:

docker network create micro-net

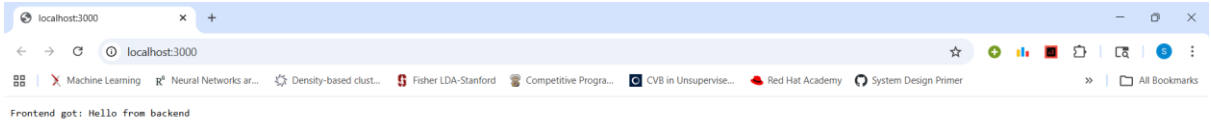
docker run --network micro-net ...

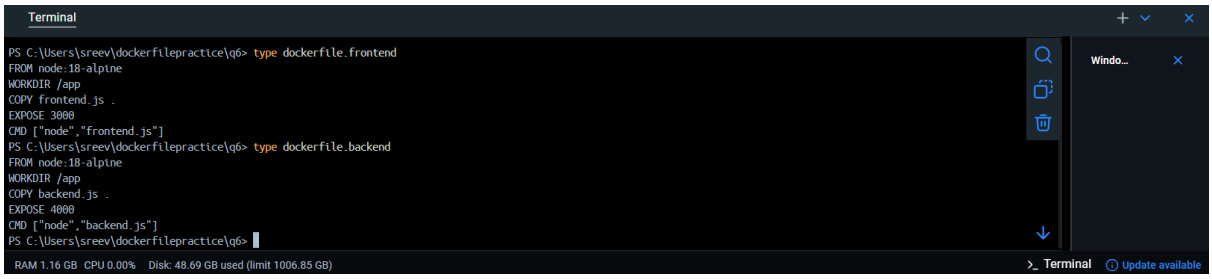
```
Terminal

PS C:\Users\sreev\dockerfilepractice\q6> notepad frontend.js
PS C:\Users\sreev\dockerfilepractice\q6> notepad backend.js
PS C:\Users\sreev\dockerfilepractice\q6> notepad dockerfile.frontend
PS C:\Users\sreev\dockerfilepractice\q6> notepad dockerfile.backend
PS C:\Users\sreev\dockerfilepractice\q6> ls


    Directory: C:\Users\sreev\dockerfilepractice\q6


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        19-04-2026     20:00            163 backend.js

RAM 1.20 GB  CPU 2.42%   Disk: 48.69 GB used (limit 1006.85 GB)
```

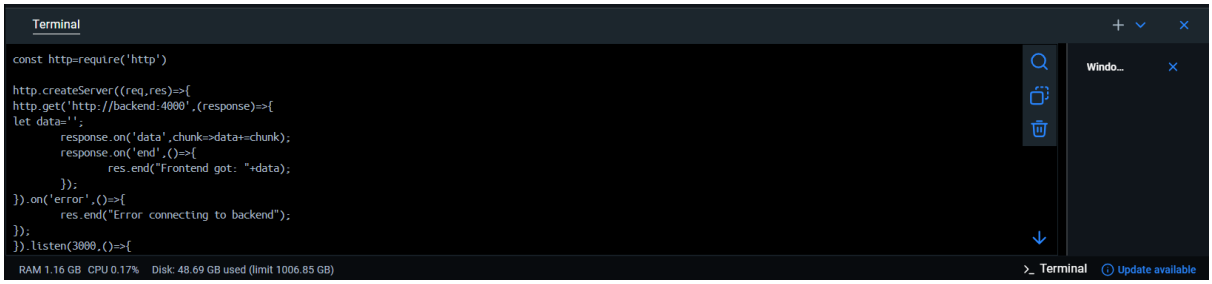
```
Terminal

PS C:\Users\sreev\dockerfilepractice\q6> docker build -t backend:v1 -f dockerfile.backend .

[+] Building 3.9s (9/9) FINISHED                                   docker:desktop-linux
 => [internal] load build definition from dockerfile.backend                        0.2s
 => => transferring dockerfile: 139B                                                0.1s
 => [internal] load metadata for docker.io/library/node:18-alpine                   2.6s
 => [auth] library/node:pull token for registry-1.docker.io                         0.0s
 => [internal] load .dockerignore                                                   0.1s
 => => transferring context: 2B                                                     0.0s
 => [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.1s
 => [internal] load build context                                                   0.1s
 => => transferring context: 202B                                                   0.1s

RAM 1.20 GB  CPU 3.77%   Disk: 48.69 GB used (limit 1006.85 GB)
```

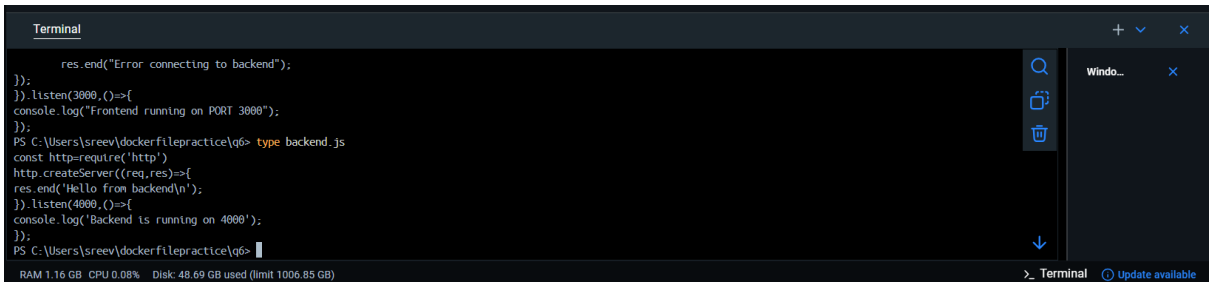
```
Terminal

PS C:\Users\sreev\dockerfilepractice\q6> docker build -t frontend:v1 -f dockerfile.frontend .
[+] Building 2.2s (8/8) FINISHED                                   docker:desktop-linux
 => [internal] load build definition from dockerfile.frontend                       0.1s
 => => transferring dockerfile: 140B                                                0.1s
 => [internal] load metadata for docker.io/library/node:18-alpine                   1.1s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.0s
 => [internal] load build context                                                   0.0s
 => => transferring context: 411B                                                   0.0s
 => CACHED [2/3] WORKDIR /app                                                       0.0s
 => [3/3] COPY frontend.js .                                                        0.1s

RAM 1.20 GB  CPU 9.31%   Disk: 48.69 GB used (limit 1006.85 GB)
```

```
Terminal

PS C:\Users\sreev\dockerfilepractice\q6> docker network create micro-net
5d9c41f602c20d6c1b041450c1d7634e7c8d5ce187e93436c5096550bbd53050
PS C:\Users\sreev\dockerfilepractice\q6> docker run -d -p 3000:3000 --name frontend frontend:v1
f16906dcc2b98e2d4b55c870053cf26cfda095b303058b6f5f825659fa8a8b9d
PS C:\Users\sreev\dockerfilepractice\q6> docker stop frontend
frontend
PS C:\Users\sreev\dockerfilepractice\q6> docker rm frontend
frontend
PS C:\Users\sreev\dockerfilepractice\q6> docker run -d -p 3000:3000 --name frontend --network micro-net frontend:v1
c4684aebc3946ba54fdbf968acc2b13426450cf2c927a74252f7dca7908c4a8d
PS C:\Users\sreev\dockerfilepractice\q6> docker run -d --name backend --network micro-net backend:v1
792ea87bb584485392527 2ab0a4dfd813ecafef58394bae969d9d78a70131a22
PS C:\Users\sreev\dockerfilepractice\q6>

RAM 1.20 GB  CPU 7.74%   Disk: 48.69 GB used (limit 1006.85 GB)
```









7. Persistent Storage for Application Data

A containerized application stores user uploads that must remain even after container deletion.

Design a storage solution using Docker volumes or bind mounts.

Students must:

- Create volume
- Attach volume to container

Commands example:

docker volume create appdata

docker run -v appdata:/data ...

```
Terminal
PS C:\Users\sreev\dockerfilepractice\q7> docker volume create mydata
mydata
PS C:\Users\sreev\dockerfilepractice\q7> docker run -it --name test-1 -v mydata:/data alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
4e0cb4ac63e7: Download complete
a63813f19ccb: Download complete
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
/ # cd data
/data # echo "lets check volume persistence">file.txt
/data # exit
PS C:\Users\sreev\dockerfilepractice\q7> docker rm test-1
RAM 1.11 GB  CPU 0.58%  Disk: 48.69 GB used (limit 1006.85 GB)
Windo...
Terminal  Update available
```

```
PS C:\Users\sreev\dockerfilepractice\q7> docker run -it --name test-2 -v mydata:/data alpine sh
/ # cd data
/data # ls
file.txt
/data # cat file.txt
lets check volume persistence
/data #
RAM 1.11 GB  CPU 0.50%  Disk: 48.69 GB used (limit 1006.85 GB)
Terminal  Update available
```

8. Copy-on-Write Behavior Investigation

A developer modifies files inside a running container and expects new containers to have those changes.

Design an experiment demonstrating Docker's copy-on-write filesystem behavior.

Students must:

- Modify files in a running container
- Start a new container
- Compare results

```
Terminal
PS C:\Users\sreev\dockerfilepractice> mkdir q8

    Directory: C:\Users\sreev\dockerfilepractice

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d-----        19-04-2026     20:41                q8

PS C:\Users\sreev\dockerfilepractice> cd q8
PS C:\Users\sreev\dockerfilepractice\q8> docker run -it --name cowpract alpine sh
RAM 1.15 GB  CPU 0.00%  Disk: 48.69 GB used (limit 1006.85 GB)
Windo...
Terminal  Update available
```

```
Terminal
PS C:\Users\sreev\dockerfilepractice\q8> docker start -ai cowpract
/ # cat /etc/hostname
e05631f7214c
/ # echo "modifying releases">/etc/alpine-release
/ # exit
PS C:\Users\sreev\dockerfilepractice\q8> docker start -ai cowpract
/ # cat /etc/alpine-release
modifying releases
/ # exit
PS C:\Users\sreev\dockerfilepractice\q8> docker run -it --name proof alpine sh
/ # cat /etc/alpine-release
3.23.4
/ #
RAM 1.15 GB  CPU 6.72%  Disk: 48.69 GB used (limit 1006.85 GB)
Windo...
Terminal  Update available
```

- **Modifying a file from the image triggers copy-on-write: Docker copies the file from the read-only image layer to the container's writable layer and applies changes there. These changes are isolated to that container and are not visible to new containers created from the same image.**

9. Image Versioning Strategy

A company releases multiple versions of its application. Developers must deploy specific versions when needed.

Design a Docker tagging and versioning strategy.

Students must:

- Build images with version tags

Commands example:

docker build -t companyapp:v1 .

docker tag companyapp:v1 companyapp:v2

10. Secure Image Distribution

A company wants to distribute container images privately among internal teams. Design a workflow using Docker Hub or GitHub Container Registry with authentication tokens.

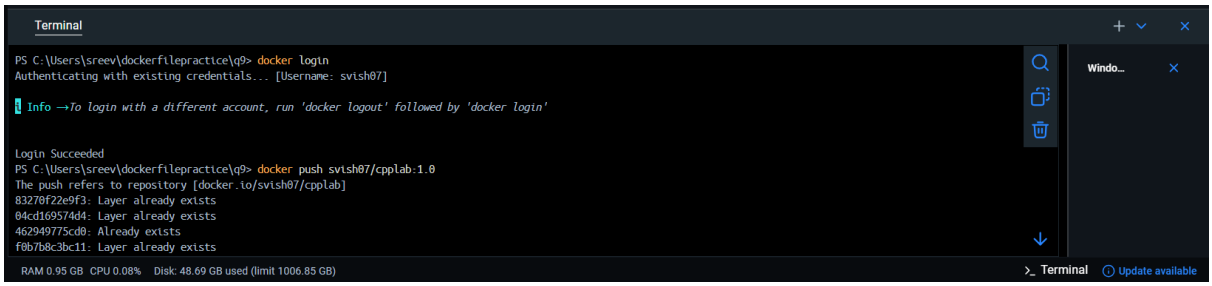
Students must:

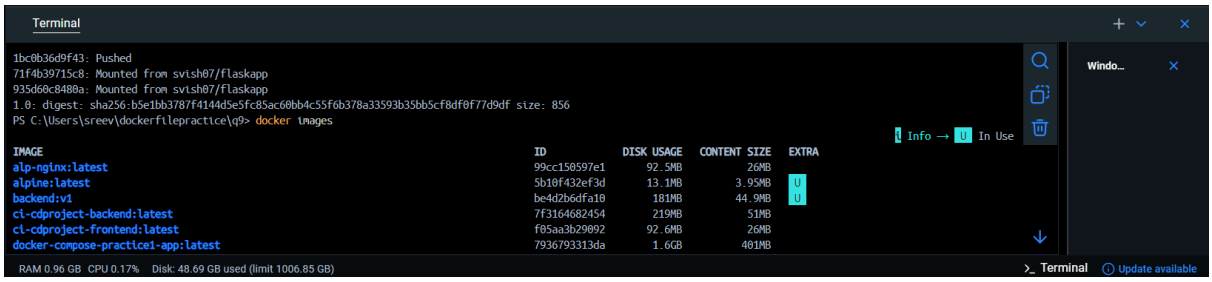
- Login to registry
- Tag image
- Push image

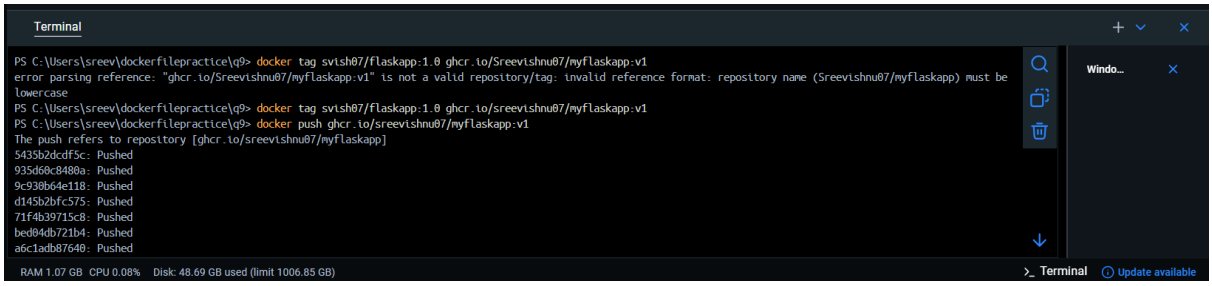
Commands example:

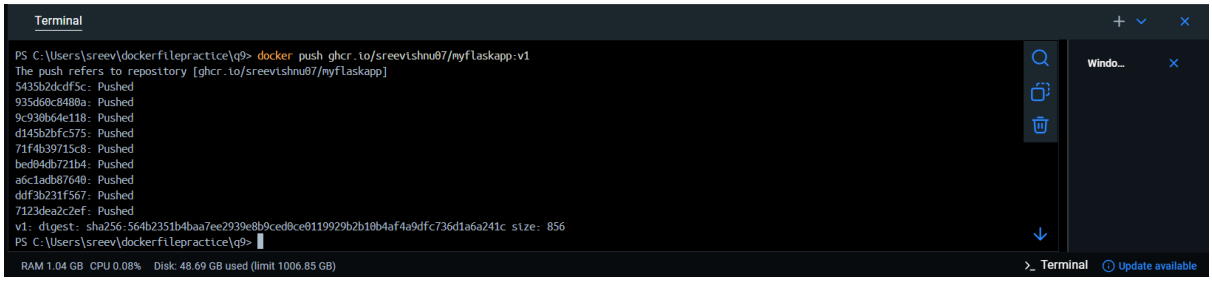
docker login

docker push username/app:v1









- **1st two contains practical on DockerHub push and last 2 images for GHCR.**